A házi feladatokról

A laborok programozásos része (házi feladatok) során egy komplexebb szoftvert kell elkészíteni.

Az eheti feladat egy determinisztikus véges automata (DFA) szimulációjának megvalósításáról szól. A későbbiekben további funkciókkal bővítjük az automata képességeit. Minden feladat parancssori paraméterekkel lesz konfigurálható, így a program működése rugalmasan szabályozható.

A feladatok kiértékelése egy automatikus rendszerrel történik, amely **minden egyes** pusholás után lefut. Ezért kiemelten fontos a pontos megvalósítás és a megfelelő kimenetek biztosítása. A legkisebb eltérés is hibás teszteredményekhez vezethet!

A projekt indulópontja a kötelezően használandó `project` modul vagy `main.cpp` fájl lesz (érdemes az elején eldönteni és utána abban dolgozni, az esetleges menet közbeni nyelvváltás nem ajánlott, mivel a feladatok egymásra épülnek, ezért újra kell implementálni az addigi feladatokat - amennyiben mégis erre kerülne sor, ezt jelezd). A létrehozott starter projektet szabadon ki lehet bővíteni, de a főfájl megnevezése nem változtatható. Minden egyes pusholáskor az automatikus tesztek lefutnak, így többször is ellenőrizheted a program helyes működését. Az összesítő eredmények a GitHub felületén, a commitok mellett jelennek meg: zöld pipa jelzi a sikeres tesztelést, piros X pedig a hibát. Hibás teszt esetén a részletes logokban megtalálható, hogy pontosan mi okozta a problémát, így a visszajelzés alapján könnyen javítható a kód.

A feladatok manuális átvizsgáláson is átesnek, hogy kiszűrjük az esetleges csalásokat vagy másolásokat.

Nulladik lépések

1. Forkold le a repositoryt.
2. Github weboldalon -> `Actions` tab -> `I understand my workflows, go ahead and enable them`

Alapvetően amikor egy workflow-t tartalmazó repositoryt leforkolsz, az actionök ki vannak kapcsolva, újra kell őket aktiválni. Ezek segítségével fogjuk tudni megcsinálni az automatikus tesztelést.

Starter projektek

Starter projektre azért van szükség, hogy egységes struktúrájú projektje legyen mindenkinek, ezzel lehetővé téve az azonnali automatikus tesztelést. Jelenleg két nyelv (template) közül lehet választani: C++ vagy Python.

A projekt leklónozása után töröljük ki az összes starter projektet, kivéve azt, amit választottunk. Ha például Pythonra esik a választás, töröljük ki az összes olyan mappát, amely nem a `python-starter` mappa.

Python nyelv esetén

Python esetén a `python-starter` nevű mappa tartalmaz egy kiinduló projektet, ami használható a feladatok megoldására.

A repository tartalmaz egy `project` nevű modult, amelyben van egy `problems` nevű almodul. Mindenképp tartsuk meg ezt a mappaszerkezetet, hogy az automatikus tesztek lefussanak.

Amennyiben a saját kódunkat akarjuk hozzáadni a projekthez, szükséges egy újabb fájl létrehozása a `problems` mappában tetszőleges néven. Ahhoz, hogy a rendszerbe beintegrálódjon a megoldásunk, fontos, hogy az új fájl tartalmazzon egy osztályt, amelyet a `Problems` osztályból származtatunk. Az almodul tartalmaz egy példa fájlt (feladatot) erre, amely bemutatja hogyan lehet ezt kivitelezni: ez lesz a `sum.py`, amely beolvassa az argumentumokat, majd összeadja azokat (a számok vesszővel vannak elválasztva). A **bemeneti fájlok** a `python-starter` mappába kell kerüljenek.

A program és a teljes start projekt el van látva kommentekkel, amelyek segítenek a megértésében és a benne való eligazodásában.

Ha nem szeretnénk használni a megadott sablont, helyettesíthetjük bármivel. A fontos az, hogy maradjon meg a `project` nevű modul, és futtatható legyen a következő paranccsal:

```
python -m project
```

C++ nyelv esetén

C++ esetén a `cpp-starter` nevű mappa tartalmaz egy kiinduló projektet, ami használható a feladatok megoldására.

Mindenképp tartsuk meg a `CMakeLists.txt` állományt, hogy az automatikus tesztek lefussanak.

Amennyiben a saját kódunkat akarjuk hozzáadni a projekthez, szükséges egy új cpp/hpp (`.hpp` a C++ “header fájlja”) pár létrehozása a `problems` mappában tetszőleges néven. Ahhoz, hogy a rendszerbe beintegrálódjon a megoldásunk, fontos, hogy az új fájl taralmazzon egy osztályt, amelyet a `Problem` osztályból származtatunk (`problem.hpp` headerből). Az osztályunkat be kell helyezni a `main.cpp`-ben a `problems` nevű vektorba. A kiinduló projekt tartalmaz egy példa feladatot erre, amely bemutatja hogyan lehet ezt kivitelezni: ez lesz a `sum.cpp` és a `sum.hpp` állomány, amely beolvassa az

argumentumokat, majd összeadja azokat (a számok szóközzel vagy új sorral vannak elválasztva). A **bemeneti fájlok** a `build` mappába kell kerüljenek.

A program és a teljes start projekt el van látva kommentekkel, amelyek segítenek a megértésében és a benne való eligazodásában.

Ha nem szeretnénk használni a megadott sablont, helyettesíthetjük bármivel. A fontos az, hogy maradjon meg a `CMakeLists.txt` állomány, és futtatható legyen a projekt a következő parancssorozattal:

```
mkdir build
cd build
cmake ..
make -j$(nproc)
./project
```

1. feladat: Determinisztikus véges automata szimulációja

Leírás

Írj egy programot, amely egy **determinisztikus véges automata** (DFA, *deterministic finite automaton*) működését szimulálja. A program egy szövegállományból olvassa be az automata adatait, majd tetszőleges számú szót vizsgál meg, hogy az automata felismeri-e őket. Amennyiben egy szó eleme az automata által felismert nyelvnek, a program írja ki a kimeneti fájlba, hogy **“IGEN”** , máskülönben, hogy **“NEM”** (nagy betűkkel) - ha több szó is van, amit tesztelni kell, akkor új sor válassza el ezeket a szavakat.

Feladat követelményei

1. A programnak képesnek kell lennie egy adott fájlból beolvasni az automata állapotait, átmeneteit, kezdő állapotát, és végállapotait.
2. A bemenetként megadott szavak alapján dönti el, hogy az automata elfogadja-e az adott szót.
    - Az eredményeket egy fájlba kell kiírni.
    - A program működése konzolról konfigurálható az alábbi parancssori argumentumokkal:
        - `--input <fájlnév>`: A bemeneti fájl, amely tartalmazza az automata adatait.
        - `--output <fájlnév>`: A fájl, amelybe az eredményeket kiírja.

- `--check <szó>`: Jelezzük a programnak, hogy egy szót akarunk ellenőrizni, és megadjuk a szót. Több szót is megadhatunk, ez esetben vesszővel (szóközök nélkül) választjuk el őket, például: `a,ab,abc,abcd`. Ebben az esetben minden szót le kell ellenőrizni, illetve a kimenetben minden egyes szó eredményét új sorba kell írni. Vigyázzunk arra, hogy ezek nagybetűsítve legyenek.

További feladatok is lesznek a későbbiekben, amelyek más feladattípusokhoz kapcsolódnak, ezek új kapcsolót/flaget kapnak majd. Tehát egy teljes parancssor a következőképp nézne ki, ha Pythont használunk:

```
python -m project --input input_file1.txt --output output_file1.txt --check abb
```

Ha C++-t használunk, akkor a következőképp nézne ki a futtatása:

```
mkdir build
cd build
cmake ..
make -j$(nproc)
./project --input input_file1.txt --output output_file1.txt --check abb
```

Bemenet

A bemeneti fájl az alábbi formátumot követi:

- **Első sor** : Állapotok, szóközökkel elválasztva
- **Második sor** : Bemeneti ábécé elemei, szóközökkel elválasztva
- **Harmadik sor** : Kezdőállapot
- **Negyedik sor** : Végállapotok, szóközökkel elválasztva
- **Következő sorok** : Egy-egy átmenet a következő formátumban: , szóközökkel elválasztva, ahol is az elsőből indulunk ki és eljutunk a -n keresztül a második állapotba.

A bemenet helyességét nem szükséges leellenőrizni, így a feldolgozása tetszőleges módon megvalósítható. Az ábécé elemeit például ki lehet nyerni az átmenetekből is. A formátum megtartása erúsen javasolt a későbbi automatikus tesztelés céljából.

Példa bemeneti állományra

```
q0 q1 q2
```

```
0 1 2
q0
q0
q0 0 q0
q0 1 q1
q0 2 q2
q1 0 q1
q1 1 q2
q1 2 q0
q2 0 q2
q2 1 q0
q2 2 q1
```

Tesztek

1. automata esetén

Bemenet:

```
q0 q1 q2
0 1
q0
q0
q0 0 q2
q0 1 q1
q1 0 q2
q1 1 q0
q2 0 q1
q2 1 q2
```

Tesztek:

Teszteset	Eredmény
10101	IGEN
111	NEM
111110111010101	IGEN

Teszteset	Eredmény
001	IGEN
0021	NEM

2. automata esetén

Bemenet:

```
q0 q1 q2
a b
q0
q1 q2
q0 a q1
q1 a q1
q1 b q2
q2 b q2
```

Tesztek:

Teszteset	Eredmény
a	IGEN
aa	IGEN
abab	NEM
bbb	NEM
aaaaaaaaaaaab	IGEN
aaaaabbbbb	IGEN
aaaabbbbba	NEM
c	NEM